

CRUD Operations Using Fetch API in a Node.js Application

Add, Update, Delete Student Records Using Fetch with Bootstrap Modals

1. Overview

This tutorial demonstrates how to implement **Create, Update, and Delete** operations for **Student records** using:

- Fetch API (GET, POST, PUT, DELETE)
- Bootstrap modal forms
- Multipart form handling for image uploads
- Dynamic UI updates using JavaScript
- Twig templates (substituting EJS)

It builds upon a prior implementation where:

- All student records were fetched and displayed in a table.
- Single student details were displayed in a Bootstrap modal.
- Live search functionality was implemented.

This tutorial focuses on:

1. Adding new student records
2. Updating existing student records
3. Deleting student records

2. Preparing the Add Student Modal

HTML (Twig Template Example)

```
<!-- Add Student Modal -->
<div class="modal fade" id="addStudentModal">
  <div class="modal-dialog">
    <div class="modal-content">
      <form id="addStudentForm" enctype="multipart/form-data">
        <div class="modal-body">

          <input type="text" name="first_name" placeholder="First Name"
class="form-control" />
          <input type="text" name="last_name" placeholder="Last Name" class="form-
control" />
          <input type="email" name="email" placeholder="Email" class="form-
```

```
control" />
    <input type="text" name="phone" placeholder="Phone" class="form-control"
  />

    <input type="file" name="profile_pic" class="form-control" />

  </div>
  <button type="submit" class="btn btn-primary">Create</button>
</form>
</div>
</div>
</div>
```

Important:

- `enctype="multipart/form-data"` is required for file upload.

3. Submit Event for Add Student Form

JavaScript: Add Student (POST)

```
document.querySelector("#addStudentForm").addEventListener("submit", function (e)
{
  e.preventDefault();

  const formData = new FormData(this);

  const result = await fetch(API_URL, {
    method: "POST",
    body: formData
  });

  if (result.ok) {
    this.reset();
    const modal =
bootstrap.Modal.getInstance(document.querySelector("#addStudentModal"));
    modal.hide();
    fetchStudents(); // Refresh table
  } else {
    alert("Error creating student");
  }
});
```

Notes

- `await` requires the wrapper function to be `async`.
- If using arrow functions, use `e.target` instead of `this`.

- Successful response triggers:
 - form reset
 - modal close
 - table refresh

4. Handling Common Error: FormData Constructor Error

Error:

```
Failed to construct FormData: parameter 1 is not of type HTMLFormElement
```

Cause: Using arrow function with `this` keyword.

Fix: Use normal function:

```
document.querySelector("#addStudentForm").addEventListener("submit", function (e)
{
    const formData = new FormData(this);
});
```

5. Deleting a Student (DELETE)

JavaScript: Delete Student Function

```
async function deleteStudent(id) {
    if (confirm("Are you sure to delete this student?")) {

        await fetch(`${API_URL}/${id}`, {
            method: "DELETE"
        });

        fetchStudents(); // Refresh table
    }
}
```

Twig Template Button Example

```
<button class="btn btn-danger" onclick="deleteStudent({{ student.id
}})">Delete</button>
```

6. Editing Existing Student — Fetching Student Data

Clicking EDIT should:

1. Open a modal form
2. Fetch the selected student's data
3. Populate the form fields

Edit Student Modal (Twig)

```
<div class="modal fade" id="editStudentModal">
  <div class="modal-dialog">
    <div class="modal-content">
      <form id="editStudentForm" enctype="multipart/form-data">
        <input type="hidden" id="editStudentId">

        <input type="text" id="editFirstName" name="first_name" class="form-
control" />
        <input type="text" id="editLastName" name="last_name" class="form-control"
/>
        <input type="email" id="editEmail" name="email" class="form-control" />
        <input type="text" id="editPhone" name="phone" class="form-control" />
        <input type="file" id="editProfilePic" name="profile_pic" class="form-
control" />

        <button type="submit" class="btn btn-primary">Update</button>
      </form>
    </div>
  </div>
</div>
```

JavaScript: Fetch Student Details (GET)

```
async function editStudent(id) {
  const response = await fetch(`${API_URL}/${id}`);
  const student = await response.json();

  document.querySelector("#editStudentId").value = student.id;
  document.querySelector("#editFirstName").value = student.first_name;
  document.querySelector("#editLastName").value = student.last_name;
```

```
document.querySelector("#editEmail").value = student.email;
document.querySelector("#editPhone").value = student.phone;
document.querySelector("#editGender").value = student.gender;

const modal = new
bootstrap.Modal(document.querySelector("#editStudentModal"));
modal.show();
}
```

7. Updating Student Record (PUT)

JavaScript: Update Student Form Submit

```
document.querySelector("#editStudentForm").addEventListener("submit", async
function (e) {
    e.preventDefault();

    const id = document.querySelector("#editStudentId").value;

    const formData = new FormData(this);

    const result = await fetch(`${API_URL}/${id}`, {
        method: "PUT",
        body: formData
    });

    if (result.ok) {
        const modal =
bootstrap.Modal.getInstance(document.querySelector("#editStudentModal"));
        modal.hide();
        fetchStudents(); // Refresh table
    } else {
        alert("Error updating student");
    }
});
```

8. Key Fix for Update Route

If the record was not updating, the issue was:

Missing ID in the URL:

```
await fetch(API_URL, { method: "PUT" }) // Incorrect
```

Corrected:

```
await fetch(`${API_URL}/${id}`, { method: "PUT", body: formData });
```

9. CRUD Workflow Summary

Operation	HTTP Method	Modal/Form	Notes
Add Student	POST	Add Student Modal	Uses <code>FormData</code> + <code>multipart/form-data</code>
View Student	GET	View Modal	Auto-populated via Fetch
Edit Student	GET (fetch), PUT (update)	Edit Student Modal	Hidden field for ID
Delete Student	DELETE	Button action	Confirmation dialog